# BÁO CÁO BÀI TẬP LỚN
# MÔN: THỊ GIÁC MÁY TÍNH

## ĐỀ TÀI: OCR – NHẬN DIỆN VĂN BẢN TRONG HÌNH ẢNH BẰNG PYTHON

Sinh viên: ..... Mã sinh viên: .....

Lớp: ..... Giảng viên: .....

## MỞ ĐẦU

Trong thực tế, thông tin thường được lưu trữ dưới dạng hình ảnh như ảnh chụp tài liệu, hóa đơn, biểu mẫu và biển báo. Việc nhập lại thủ công nội dung từ các hình ảnh này tốn thời gian và dễ phát sinh sai sót. Công nghệ OCR (Optical Character Recognition) hỗ trợ tự động phát hiện và chuyển đổi vùng chữ trong ảnh thành văn bản có thể chỉnh sửa, tìm kiếm và lưu trữ.

Đề tài xây dựng ứng dụng OCR trên máy tính bằng Python. Hệ thống sử dụng OpenCV để tiền xử lý ảnh, EasyOCR để phát hiện và nhận diện văn bản, Tkinter để xây dựng giao diện. Ngoài nhận diện từ ảnh có sẵn, chương trình hỗ trợ webcam, hiển thị ảnh trước/sau xử lý, đánh giá CER/WER và xuất báo cáo.

## CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Mục tiêu: xây dựng ứng dụng desktop có khả năng nhận diện văn bản tiếng Việt và tiếng Anh từ ảnh; hỗ trợ tiền xử lý; nhận ảnh từ webcam; đánh giá kết quả bằng văn bản tham chiếu; lưu kết quả.

1.2. Phạm vi: hệ thống xử lý ảnh tĩnh hoặc một khung hình chụp từ webcam. Phiên bản này chưa tập trung vào nhận diện chữ viết tay phức tạp, bảng biểu nhiều cột hoặc tài liệu bị che khuất.

1.3. Đầu vào là ảnh RGB. Đầu ra gồm các vùng văn bản, chuỗi ký tự, độ tin cậy theo vùng, thời gian xử lý và các chỉ số đánh giá nếu có văn bản chuẩn.

## CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. OCR: quy trình OCR thường gồm phát hiện vùng chứa chữ (text detection), nhận dạng nội dung ký tự (text recognition) và hậu xử lý. Detection xác định vị trí văn bản; recognition chuyển vùng chữ thành chuỗi ký tự.

2.2. Grayscale: chuyển ảnh màu thành ảnh mức xám nhằm giảm số kênh dữ liệu và tập trung vào cường độ sáng.

2.3. Gaussian Blur: làm trơn ảnh và giảm nhiễu tần số cao. Kích thước kernel cần được chọn phù hợp; làm mờ quá mạnh có thể mất nét chữ.

2.4. CLAHE: cân bằng lược đồ xám cục bộ có giới hạn tương phản, giúp cải thiện vùng tối/sáng không đồng đều và hạn chế khuếch đại nhiễu quá mức.

2.5. Adaptive Threshold: xác định ngưỡng nhị phân theo vùng lân cận, phù hợp hơn ngưỡng toàn cục khi ánh sáng không đồng đều. Kết quả phụ thuộc block size và hằng số C.

2.6. EasyOCR: thư viện OCR dựa trên học sâu, cung cấp mô hình phát hiện và nhận dạng chữ. Dự án cấu hình ngôn ngữ vi và en, chạy CPU để dễ triển khai trên máy cá nhân.

2.7. CER và WER: CER đo tỷ lệ thao tác ký tự (thay thế, xóa, chèn) trên độ dài chuỗi tham chiếu. WER thực hiện tương tự ở cấp độ từ. Hai chỉ số càng thấp càng tốt. Accuracy quy đổi trong chương trình là max(0, 1 - error rate), chỉ mang ý nghĩa tham khảo.

## CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Tác nhân: người sử dụng ứng dụng.

3.2. Các chức năng: chọn ảnh; tiền xử lý; nhận diện OCR; mở và chụp webcam; xem ảnh gốc/ảnh xử lý; xem văn bản và thống kê; nhập văn bản chuẩn; tính CER/WER; xuất TXT/DOCX/PDF.

3.3. Kiến trúc: giao diện Tkinter nhận thao tác người dùng; module image_processing xử lý ảnh; ocr_engine quản lý EasyOCR; accuracy tính chỉ số; camera quản lý webcam; report_export xuất tệp. Tách module giúp dễ bảo trì và kiểm thử.

3.4. Luồng xử lý: ảnh đầu vào → chuyển xám → khử nhiễu → tăng tương phản CLAHE → nhị phân hóa thích nghi → EasyOCR → tổng hợp văn bản/độ tin cậy → hiển thị và xuất kết quả. Người dùng có thể chọn ảnh gốc làm đầu vào OCR hoặc chủ động tiền xử lý trước.

## CHƯƠNG 4. CÀI ĐẶT VÀ TRIỂN KHAI

4.1. Ngôn ngữ và thư viện: Python; OpenCV; NumPy; Pillow; EasyOCR; PyTorch; Tkinter; python-docx; ReportLab; jiwer.

4.2. Môi trường khuyến nghị: Windows 10/11, Python 3.12, môi trường ảo venv. Cài PyTorch CPU trước, sau đó cài requirements.txt. Chạy ứng dụng bằng lệnh python main.py.

4.3. Lần đầu sử dụng EasyOCR cần Internet để tải trọng số mô hình. Những lần tiếp theo thư viện sử dụng mô hình đã lưu trong cache.

4.4. Webcam: chương trình mở camera index 0, đọc khung hình, chuyển BGR sang RGB và cho phép chụp một khung hình để đưa vào quy trình OCR.

## CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ

5.1. Kiểm thử chức năng: chọn ảnh hợp lệ; hủy hộp thoại chọn ảnh; chạy OCR; tiền xử lý; bật/tắt camera; chụp ảnh; đánh giá với văn bản chuẩn; xuất TXT/DOCX/PDF; đóng ứng dụng khi camera đang chạy.

5.2. Kiểm thử chất lượng: nên chuẩn bị tối thiểu 20–30 ảnh có văn bản chuẩn, gồm ảnh rõ, ảnh ánh sáng yếu, ảnh nghiêng, ảnh có nhiễu và nhiều cỡ chữ. Lưu văn bản chuẩn tương ứng cho từng ảnh.

5.3. Cách đánh giá: chạy OCR trên từng ảnh; lưu chuỗi dự đoán; so sánh với chuỗi tham chiếu; ghi CER, WER, thời gian và độ tin cậy trung bình vào bảng kết quả. Báo cáo kết quả trung bình trên toàn bộ tập kiểm thử.

5.4. Bảng mẫu để điền số liệu thực nghiệm:

STT | Loại ảnh | CER | WER | Thời gian (giây) | Nhận xét

1 | Ảnh tài liệu rõ | Điền kết quả | Điền kết quả | Điền kết quả | ...

2 | Ảnh nghiêng | Điền kết quả | Điền kết quả | Điền kết quả | ...

3 | Ảnh thiếu sáng | Điền kết quả | Điền kết quả | Điền kết quả | ...

Không đưa số liệu giả định vào báo cáo. Kết quả thực tế phụ thuộc bộ ảnh, cấu hình máy và phiên bản mô hình.

## CHƯƠNG 6. HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

Hạn chế: ảnh mờ hoặc độ phân giải thấp làm giảm khả năng nhận dạng; tiền xử lý nhị phân không phải lúc nào cũng tốt hơn ảnh màu; OCR có thể đọc sai dấu tiếng Việt; webcam phụ thuộc driver và quyền truy cập; độ tin cậy của mô hình không đồng nhất với độ chính xác thực nghiệm.

Hướng phát triển: tự động phát hiện góc nghiêng tốt hơn; cho phép chọn nhiều ảnh và xử lý hàng loạt; lưu lịch sử vào SQLite; xuất Excel; nhận diện bảng biểu; xây dựng bộ dữ liệu đánh giá có nhãn; thử nghiệm nhiều cấu hình tiền xử lý và so sánh CER/WER.

## KẾT LUẬN

Đề tài đã thiết kế ứng dụng OCR desktop bằng Python, kết hợp xử lý ảnh OpenCV và mô hình nhận diện EasyOCR. Ứng dụng hỗ trợ ảnh tĩnh, webcam, xem ảnh trước/sau tiền xử lý, hiển thị văn bản, thống kê và xuất báo cáo. Chức năng CER/WER tạo cơ sở đánh giá định lượng khi có văn bản tham chiếu.

Qua đề tài, người thực hiện vận dụng các kiến thức thị giác máy tính như chuyển đổi không gian màu, khử nhiễu, tăng tương phản, nhị phân hóa và nhận dạng mẫu bằng học sâu. Để kết luận chất lượng hệ thống một cách khách quan, cần thực nghiệm trên tập ảnh có văn bản chuẩn và trình bày số liệu thực tế.

## TÀI LIỆU THAM KHẢO

1. OpenCV Documentation: https://docs.opencv.org/

2. EasyOCR GitHub: https://github.com/JaidedAI/EasyOCR

3. PyTorch Documentation: https://pytorch.org/docs/

4. jiwer Documentation: https://github.com/jitsi/jiwer

5. Python Tkinter Documentation: https://docs.python.org/3/library/tkinter.html